

Recap: Linear programs (LP)

An optimization problem that aims to **maximize or minimize a linear objective function**, subject to **linear equality and/or inequality constraints**.

$$\begin{array}{ll}\text{Maximize} & \mathbf{c}^T \mathbf{x} \\ \text{Subject to} & \mathbf{Ax} \leq \mathbf{b} \\ & \mathbf{x} \geq 0\end{array}$$

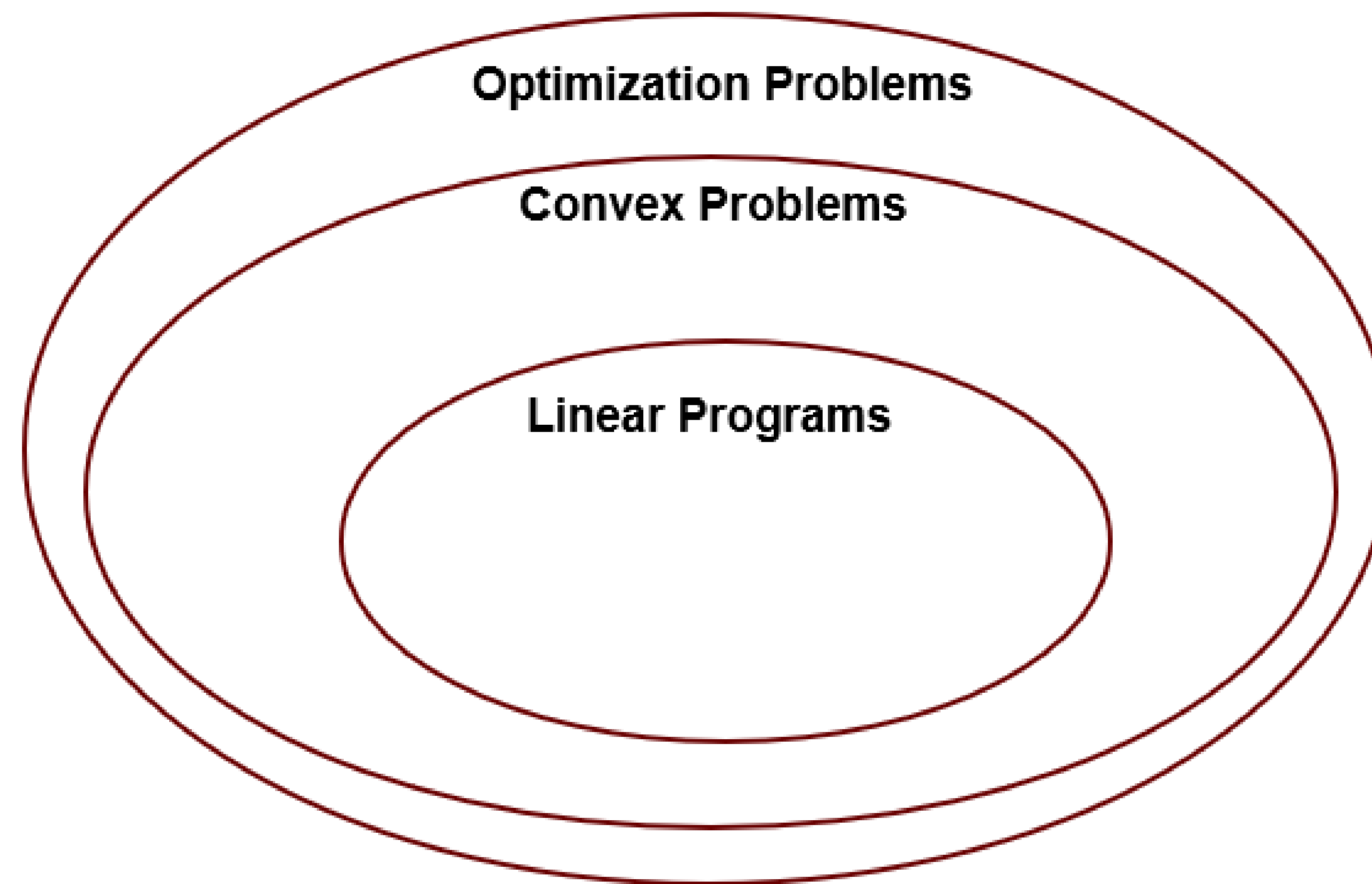
Solving LPs:

Graphical method: applicable when there are only two variables.

Simplex algorithm: efficient for large-scale LPs

Fundamental theorem in LP: Optimal solutions occur on the boundary of the feasible region — typically at its vertices (or along an edge between vertices).

Linear programs vs convex problems



Linear programming is probably the most widely studied and broadly useful method for optimization. It is a special case of the more general problem of **convex optimization**, which allows the constraint region to be any convex region and the objective to be any function that is convex within the constraint region. Under certain conditions, convex optimization problems are also polynomially solvable and may be feasible in practice with thousands of variables. Several important problems in machine learning and control theory can be formulated as convex optimization problems (see Chapter 21).

Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P140

Agenda

- **Background: Convex sets and convex functions**
- Convex optimization problems: Real-world examples
- Solving convex problems: Gradient descent

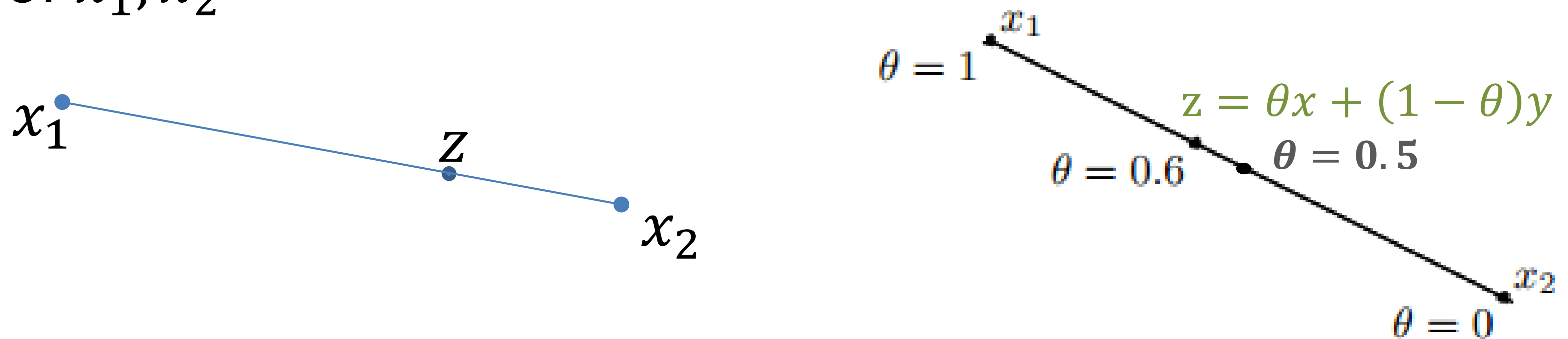


Convex Optimization
Stephen Boyd and Lieven Vandenberghe
Cambridge University Press

Reference (modern bible of convex optimization): Stephen Boyd & Lieven Vandenberghe, Convex Optimization, <https://web.stanford.edu/~boyd/cvxbook/>

Convex combination: Definition

- A point between two points
- Given $x_1, x_2 \in \mathbb{R}^n$, a *convex combination* of them is any point of the form $z = \theta x_1 + (1 - \theta)x_2$ where $\theta \in [0,1]$
- When $\theta \in (0,1)$, z is called a strict convex combination of x_1, x_2

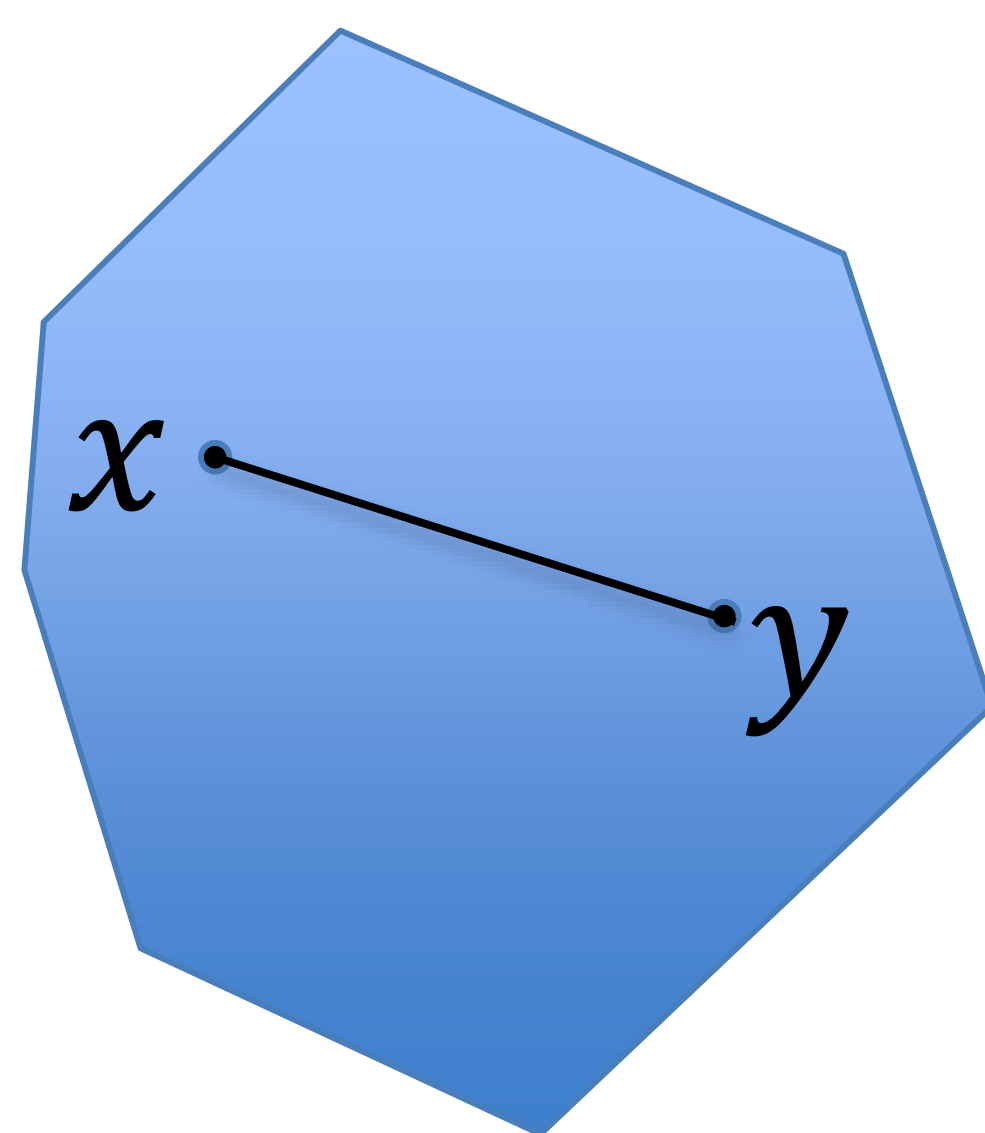


A convex combination is just a fancy way of saying "a point between two points."

Convex set: Definition

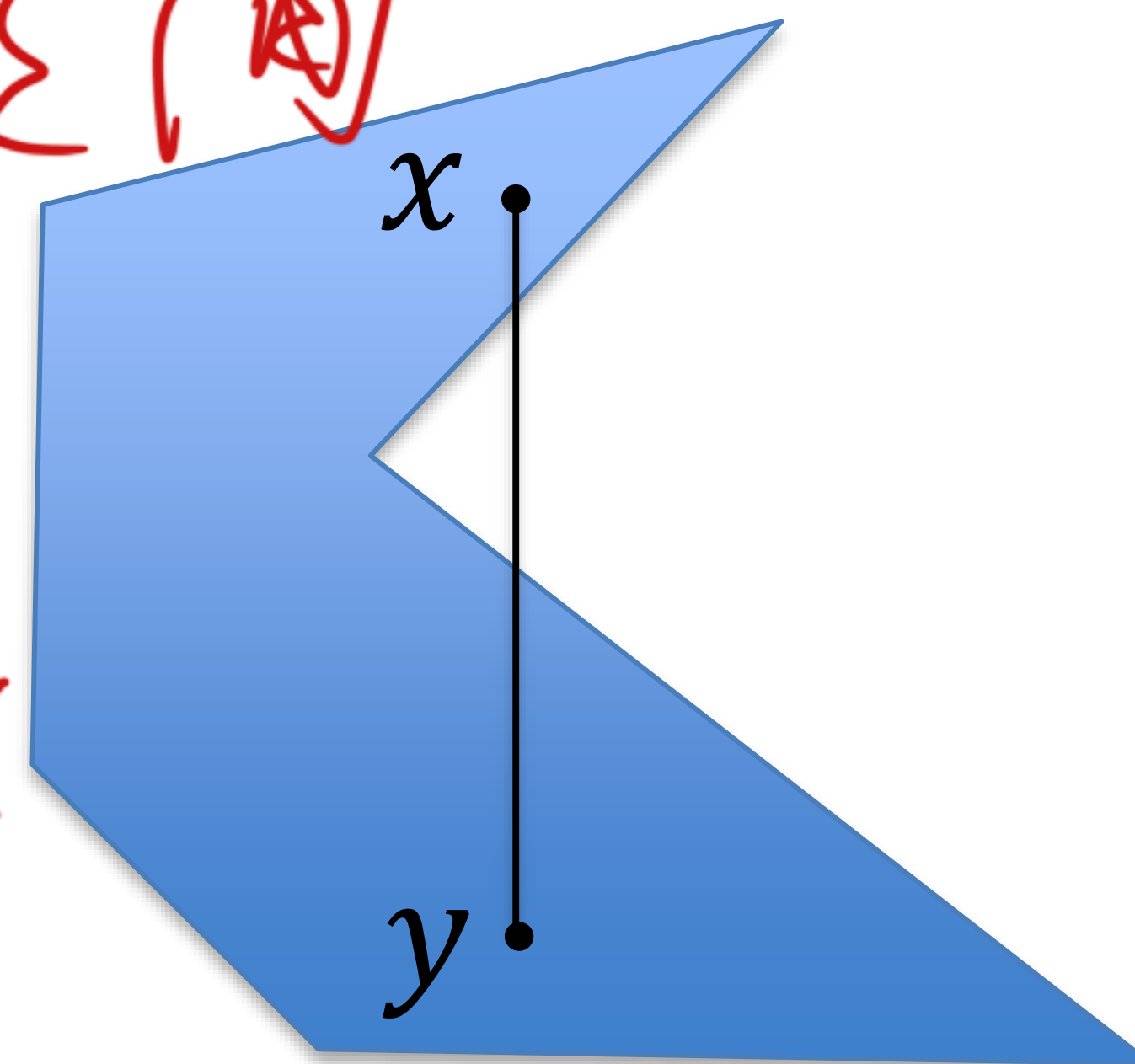
A convex set is a set $C \subseteq R^n$ such that for all $x_1, x_2 \in C$ and $\theta \in [0, 1]$ we have :

$$\theta x_1 + (1 - \theta)x_2 \in C$$



Convex set

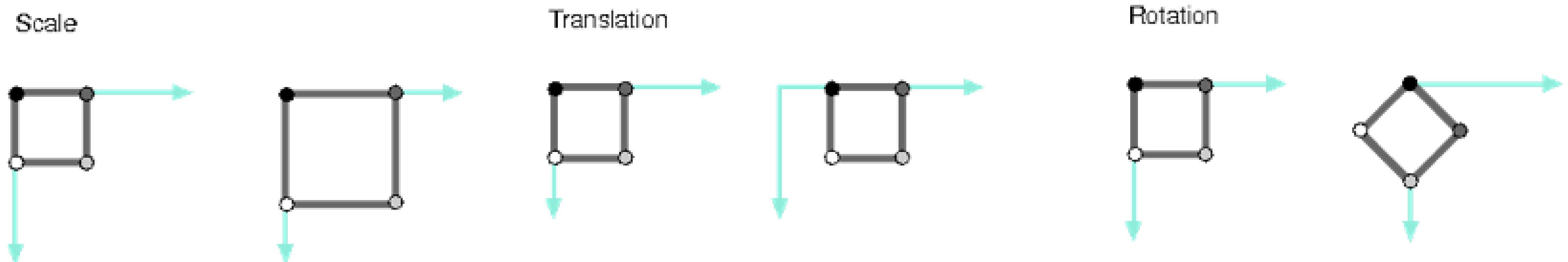
任何在 $x, y \in \text{set}$
的點都在 set
之中 $\rightarrow$ convex
set



Non-convex set

Convex set: Properties

- The empty set $\emptyset$ and $\mathbb{R}^n$ are both convex.
- Intersections of convex sets are convex.
- Convexity is **preserved** under three common operations:
 - **Scaling:** Multiplying all points by a positive scalar
 - **Translation:** Shifting to a new position (adding a constant vector to all points)
 - **Rotation:** Rotating them around an axis



Convex function: Definition

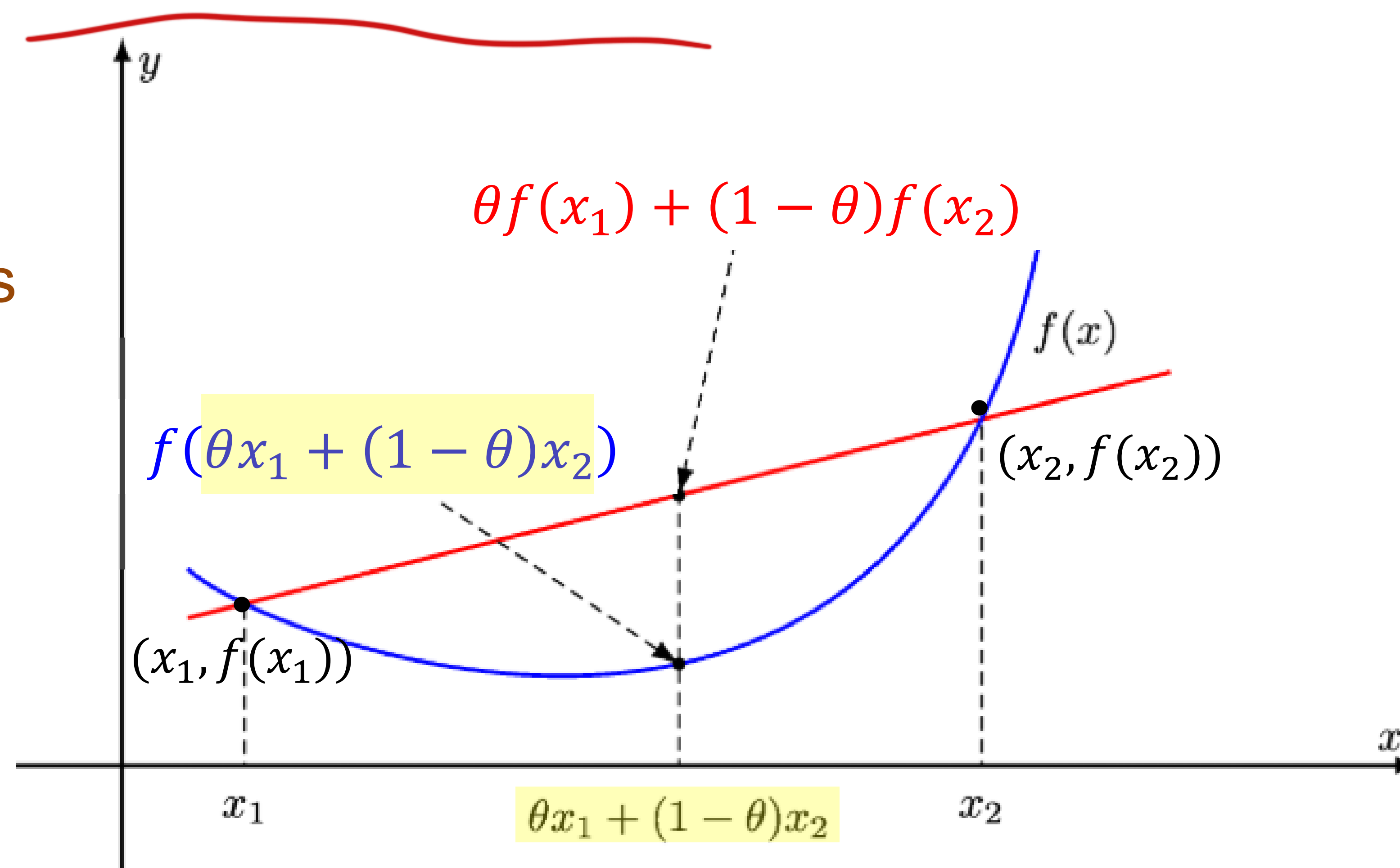
Let C be a convex set. A function $f: C \rightarrow \mathbb{R}$ is convex in C if $\forall x_1, x_2 \in C, \forall \theta \in [0,1]$,

$$f(\theta x_1 + (1 - \theta)x_2) \leq \theta f(x_1) + (1 - \theta)f(x_2)$$

If $C = \mathbb{R}^n$, we simply say f is a convex function.

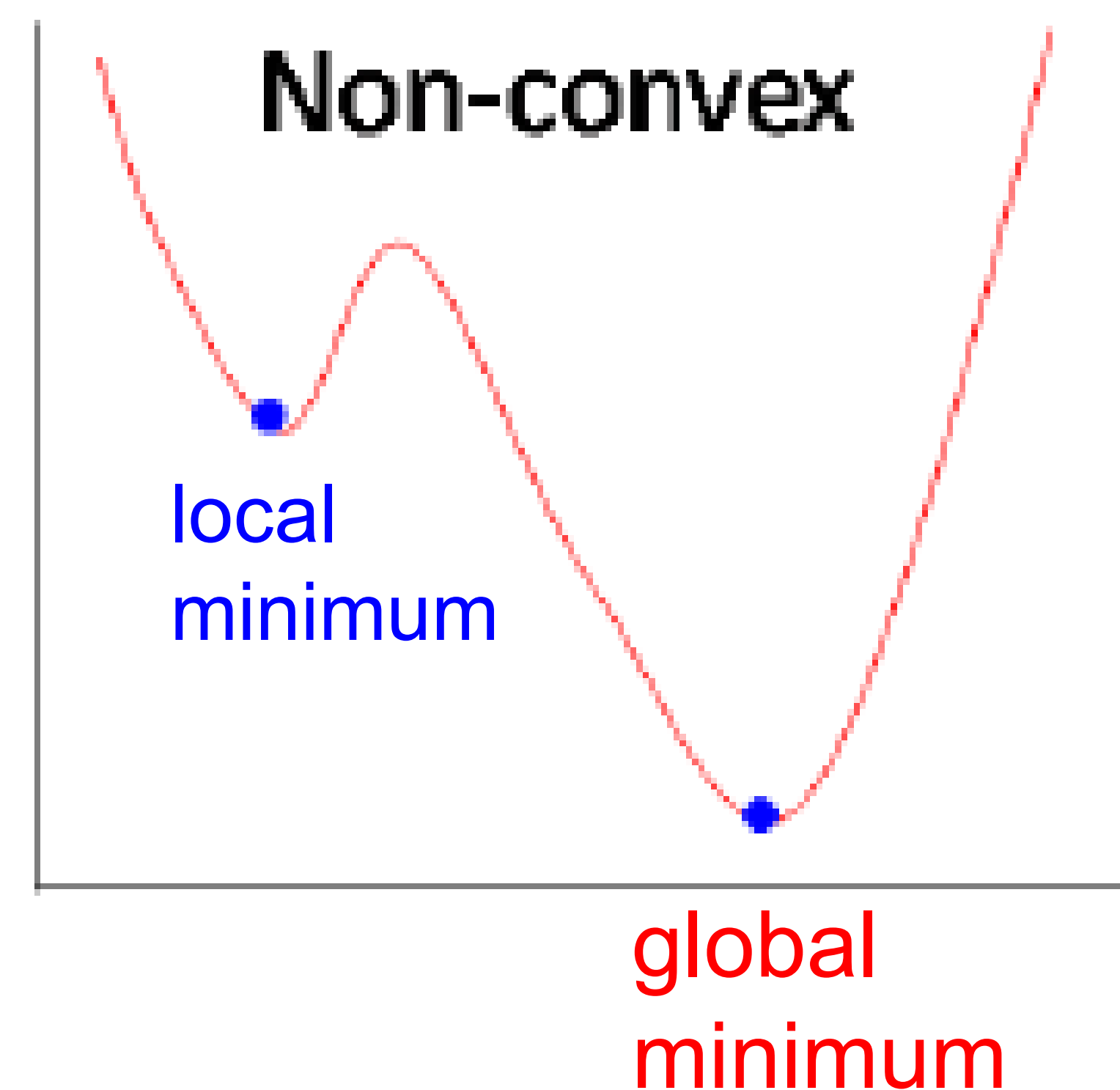
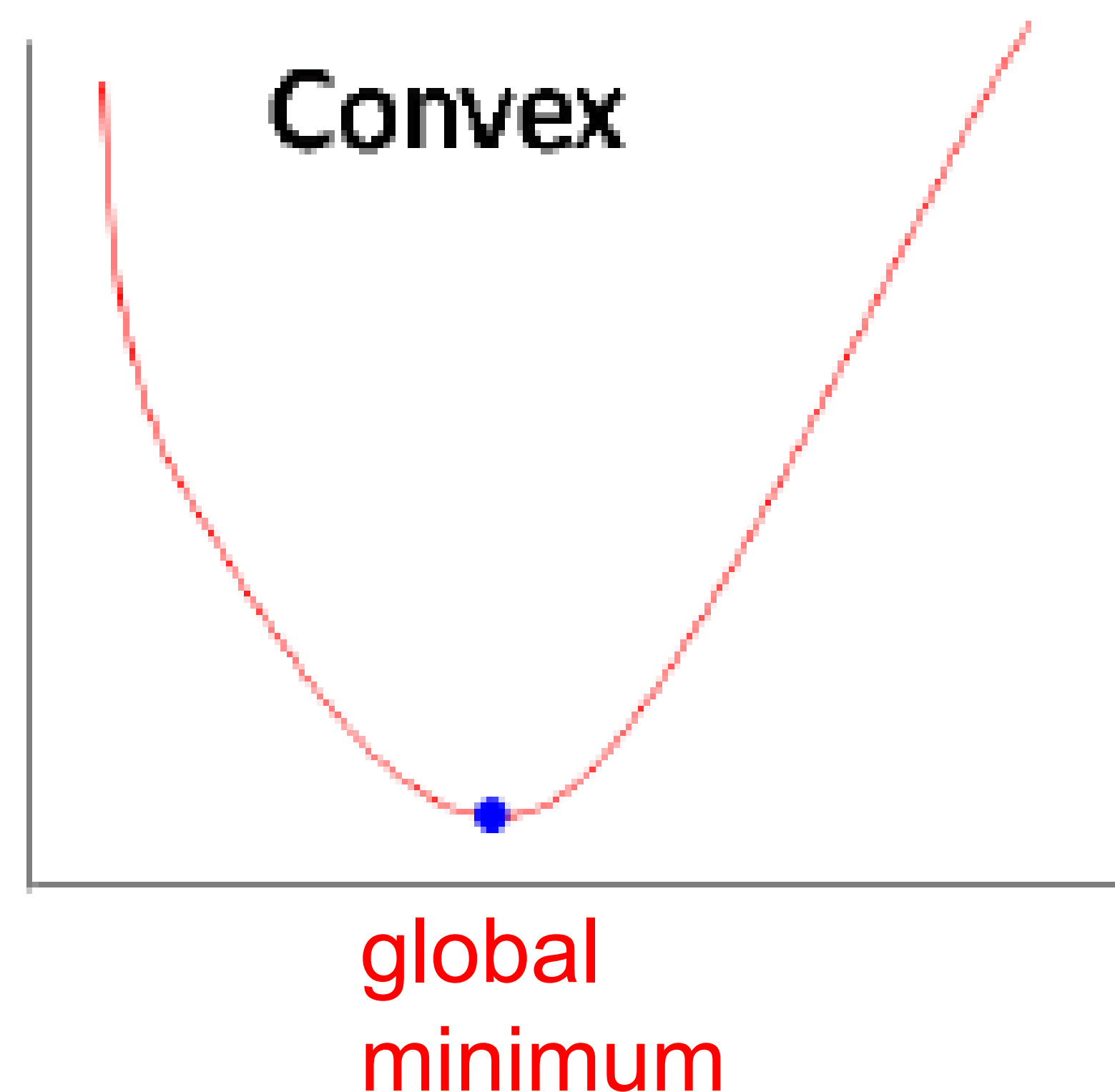
Any chord between two points on the function always lies above the function.

Value in the middle point is lower than the average of the two endpoint values.



Convex function: Properties

- Sum of convex functions is convex
 - $af(x) + bg(x)$ is convex for convex functions f, g and $a, b > 0$.
- Convexity is preserved under affine transformation
 - If $f(\mathbf{x}) = g(\mathbf{Ax} + \mathbf{b})$, g is convex, then $f(\mathbf{x})$ is convex.
 where $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^m$
- Any **local minimum** is a **global minimum**



Recap: Affine transformation

Definition:

An affine transformation is a fundamental concept in linear algebra and geometry. It's a type of transformation that preserves points, straight lines, and planes. Crucially, it also preserves parallelism (lines that are parallel before the transformation remain parallel after).

An affine transformation is a combination of a **linear transformation** (like rotation, scaling, reflection) and a **translation** (shifting).

$$\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{b}$$

where $\mathbf{A}$: matrix (rotation, scaling, etc.)

$\mathbf{b}$: vector (shift)

Why important?

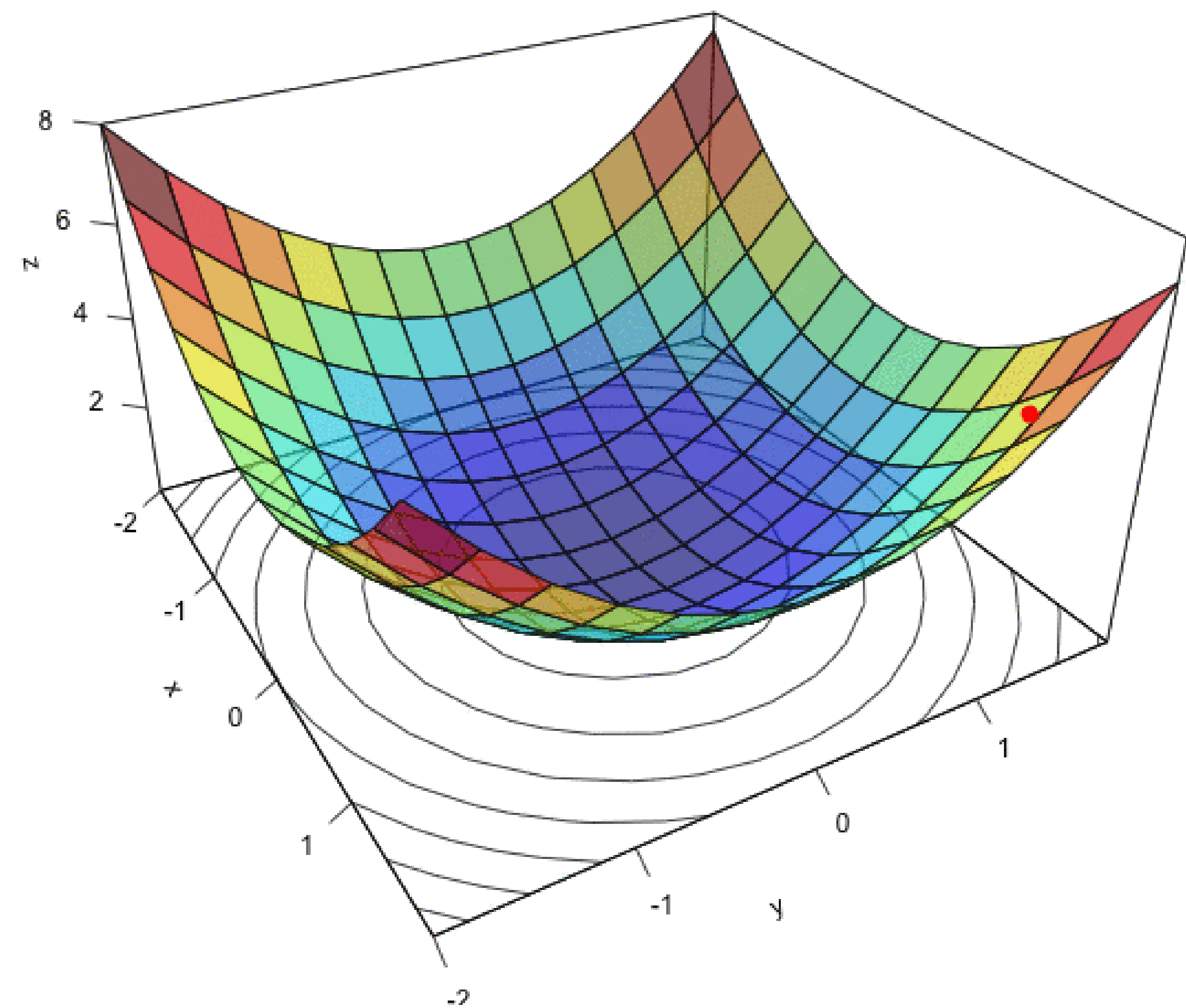
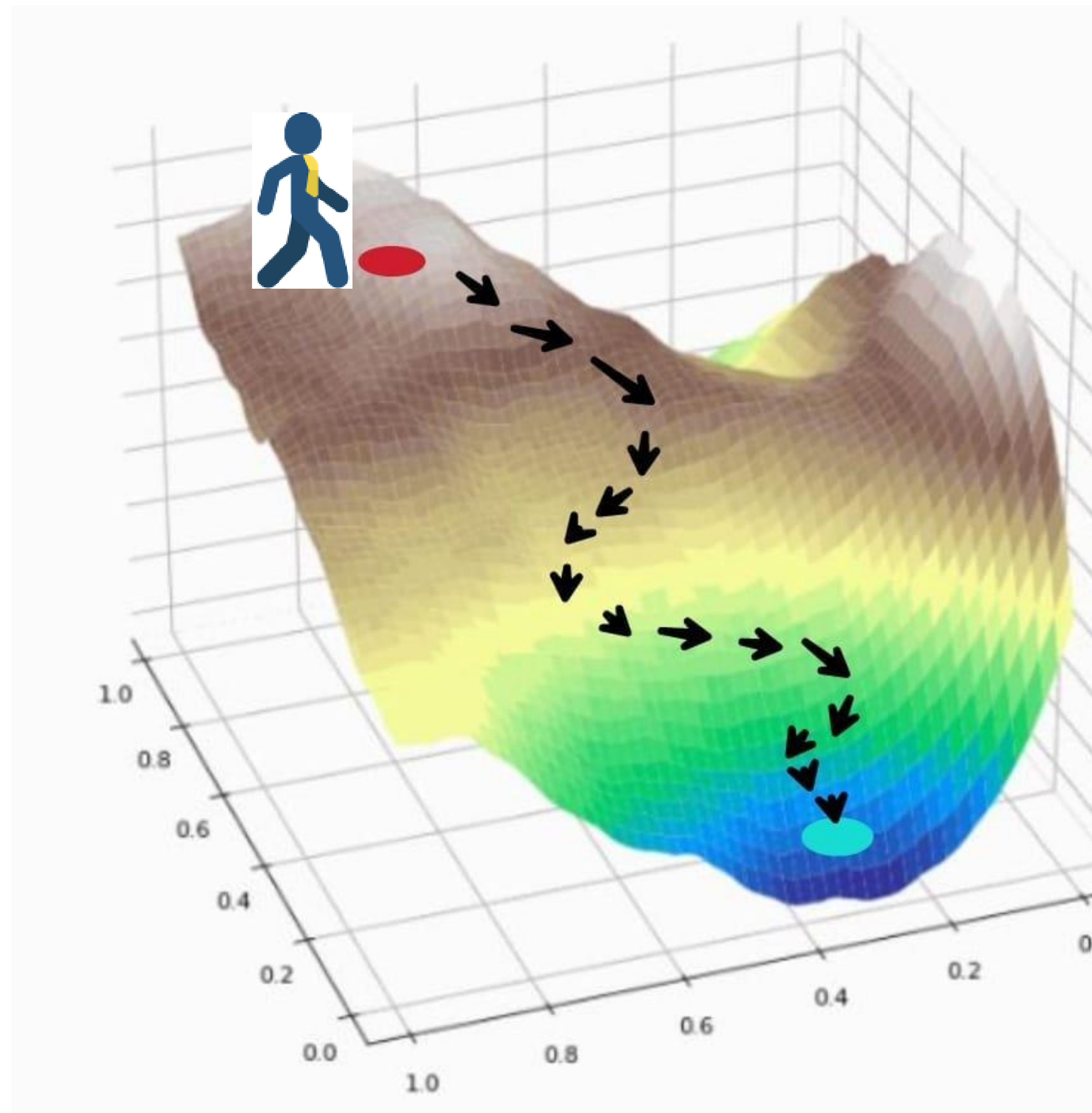
- Convexity is preserved under affine transformations.
- That means if a set or function is convex, and you apply *scaling, rotation, or translation*, it **stays convex**.

Agenda

- Background: Convex sets and convex functions
- Convex optimization problems: Real-world examples
- Solving convex problems: Gradient descent

Motivation for gradient descent

Gradient descent solves convex optimization by iteratively moving toward the global minimum along the direction of steepest decrease.

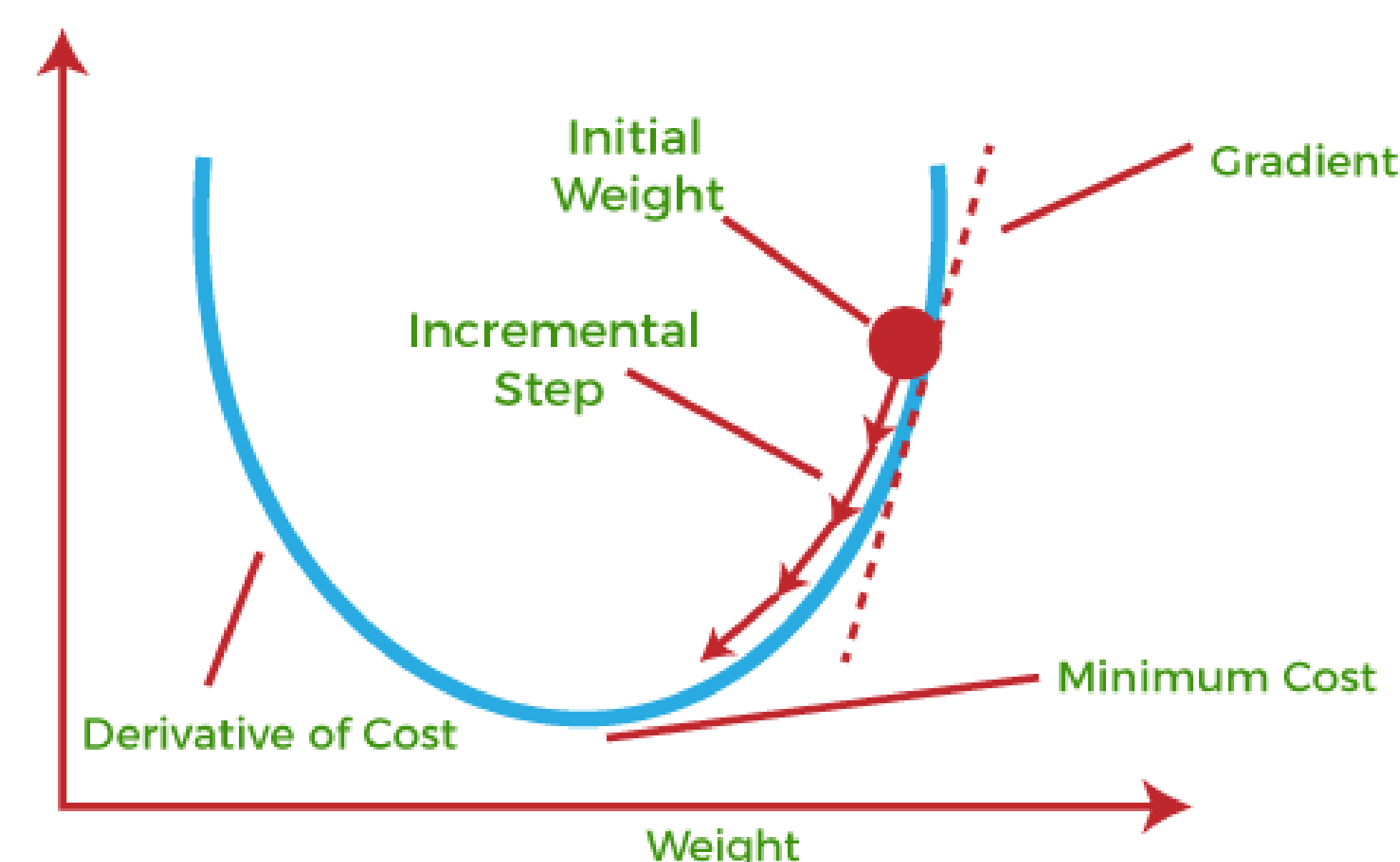


Animation sources: <https://www.analyticsvidhya.com/blog/2021/06/the-challenge-of-vanishing-exploding-gradients-in-deep-neural-networks/>

Gradient descent algorithm

Suppose we want to minimize $f(\mathbf{x})$ with respect to $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$

$$\text{Gradient } \nabla_{\mathbf{x}} f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$



$\nabla_{\mathbf{x}} f(\mathbf{x})$ is a vector and function of $\mathbf{x}$

$\nabla_{\mathbf{x}} f(\mathbf{x})$ points in the direction where f increases most rapidly.

So $-\nabla_{\mathbf{x}} f(\mathbf{x})$ is the direction of steepest descent

Gradient descent algorithm

Key points:

- $-\eta \nabla_x f(\mathbf{x}_k)$: The product of the *learning rate* and the *gradient*, which determines the *direction* and *step size* to move towards minimizing the cost.
- Common stopping criteria:
 - Maximum number of iterations is reached.
 - The percentage or absolute change in f is below a threshold.
 - The percentage or absolute change in the variable $\mathbf{x}$ is below a threshold.

Gradient Descent:

Initialize $\mathbf{x}_0$ and learning rate η

while true **do**

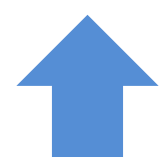
 Compute $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \eta \nabla_x f(\mathbf{x}_k)$

if converge **then**

return $\mathbf{x}_{k+1}$

end

end



Gradient descent algorithm overview

Gradient descent takes **larger steps** when the slope is **steep**, allowing for faster progress in the initial phases of optimization.

As the slope **flattens**, gradient descent takes **smaller steps**, enabling precise convergence towards the minimum ("Goal") without overshooting.

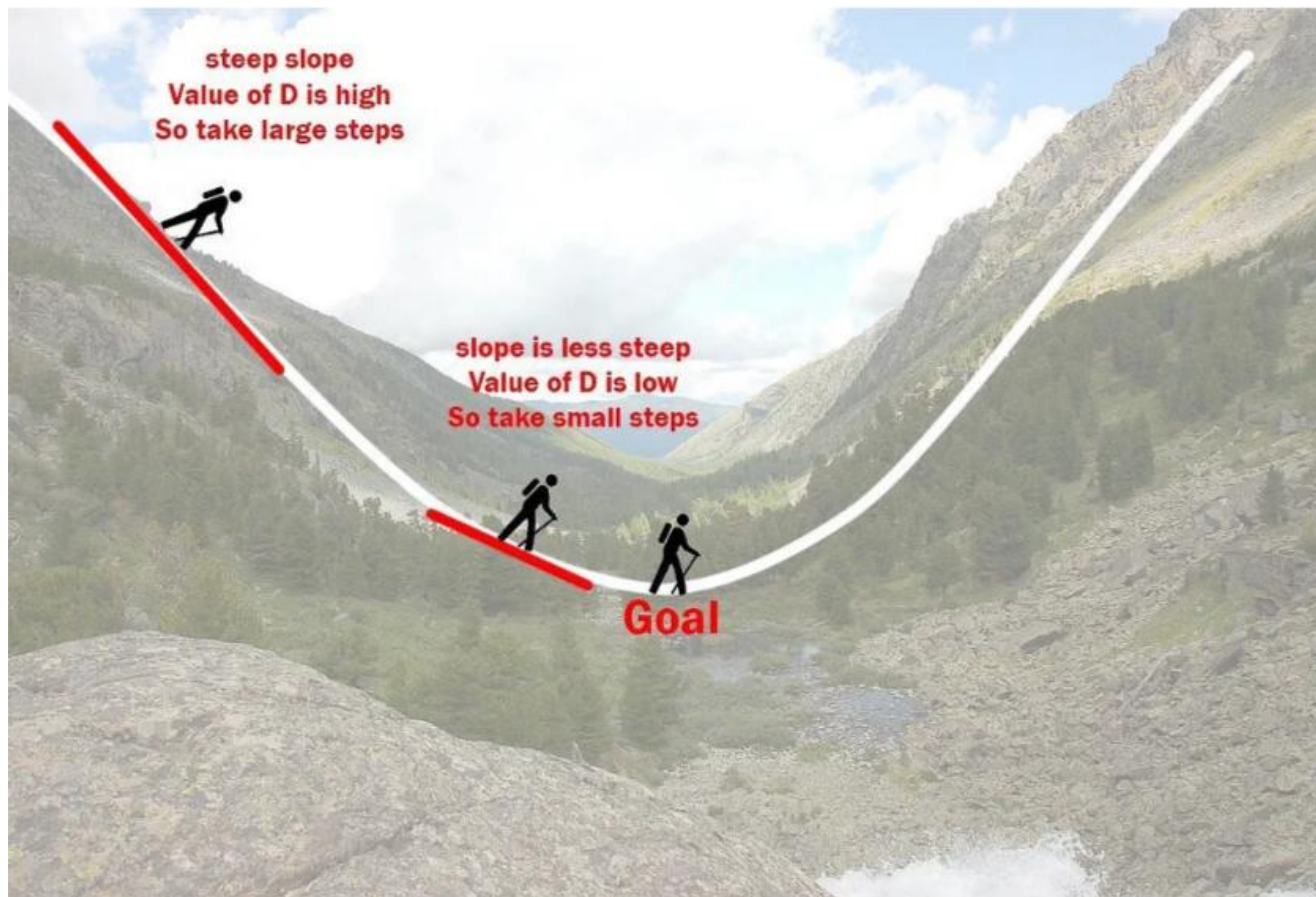


Image source: <https://www.inspiritai.com/>

Gradient descent algorithm

Choose suitable learning rate:

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$$

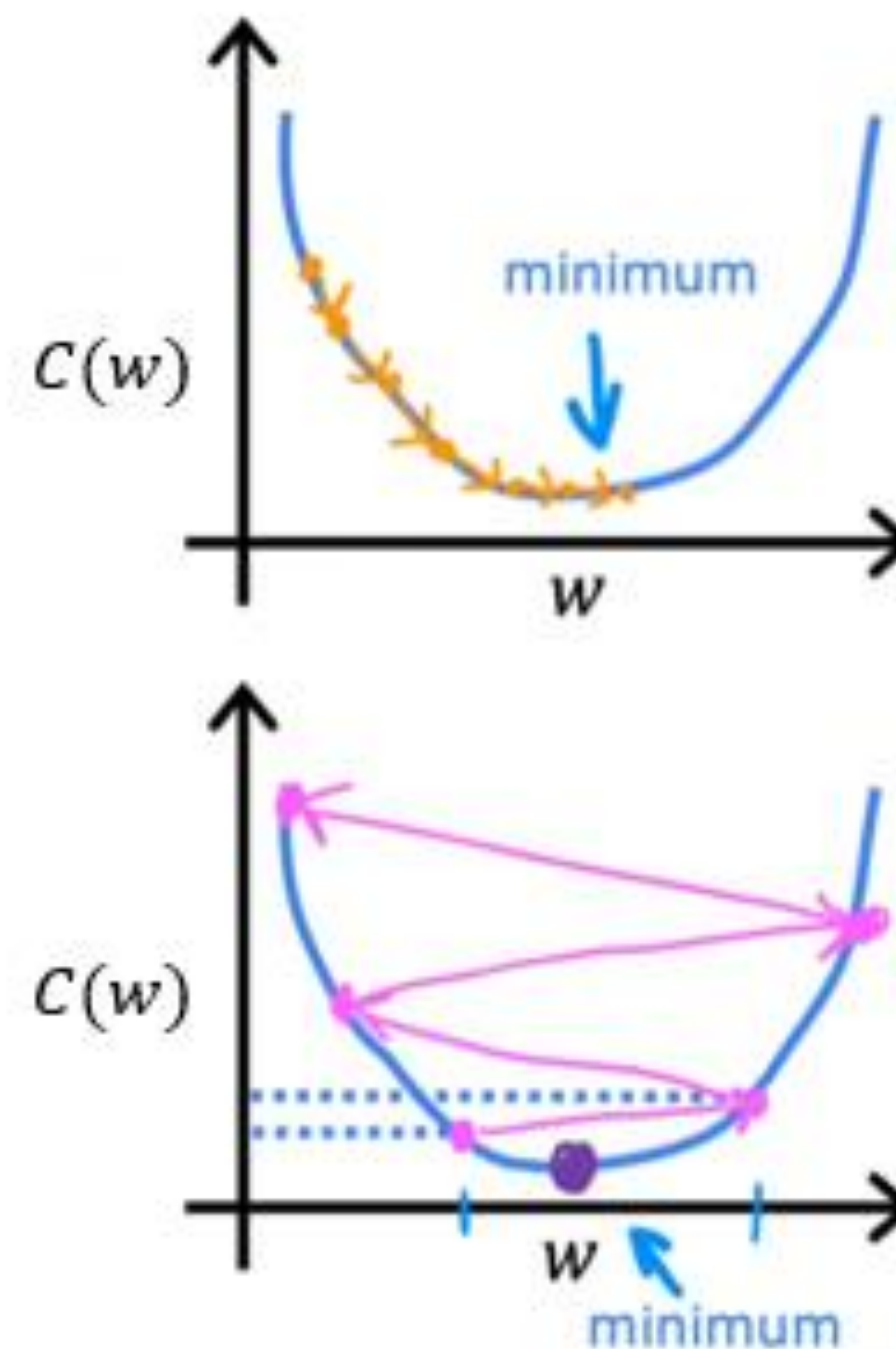
If η is too small...

Gradient descent may be slow.

If η is too large...

Gradient descent may:

- Overshoot, never reach minimum
- Fail to converge, diverge



Source: <https://www.youtube.com/watch?v=k0h8emRAAHE>

Convex optimization: How to solve

- **Unconstrained and differentiable** convex functions:
Gradient descent converges to the **global minimum** with a sufficiently small step size (i.e., learning rate η).
- **Constrained and differentiable** convex functions:
Projected gradient descent is used when the domain is constrained to a convex set.
- **Non-differentiable** convex functions (*not covered in this course*):
 ϵ -subgradient methods can be applied

* In non-convex functions, gradient descent might get stuck in local minima but sometimes finds good solutions in practice.

Projected gradient descent

Projected Gradient Descent:

Initialize $\mathbf{x}_0$ and learning rate η

while true do

 Compute:

$$\mathbf{z}_{k+1} \leftarrow \mathbf{x}_k - \eta \nabla_x f(\mathbf{x}_k)$$

$$\mathbf{x}_{k+1} \leftarrow \Pi_C(\mathbf{z}_{k+1}) \quad // \text{Project } \mathbf{z}_{k+1} \text{ onto constraint set}$$

if converge then

 | **return** $\mathbf{x}_{k+1}$

end

end

$\Pi_C(\mathbf{z})$: the projection of a point $\mathbf{z}$ onto a convex set C

$$\text{e.g., } \Pi_C(\mathbf{z}) = \underset{\mathbf{x} \in C}{\operatorname{argmin}} \|\mathbf{x} - \mathbf{z}\|_2$$

Take the point $\mathbf{z}$ (which might be outside the constraint set C), and find the *closest point* in C to it — using *Euclidean distance* (L2 norm), denoted by $\|\cdot\|$.

Projected gradient descent: Example

Qn: Minimize $g(x) = (x - 2)^2$.

Subject to: $x \geq 0$ (feasible region $\mathcal{C} = [0, +\infty)$)

Initialize $x_0 = -1$, learning rate $\eta = 0.15$

1. Compute the gradient: $\nabla_x g(x) = 2x - 4$
2. At each iteration: $x_{k+1} \leftarrow x_k - \eta \nabla_x g(x_k)$ // gradient step (tentative)
 $\mathbf{x}_{k+1} \leftarrow \Pi_{\mathcal{C}}(\mathbf{z}_{k+1})$ // projection step

Iteration 1

Gradient step (tentative):

- $x_0 = -1$
- $\nabla_x g(x_0) = 2x_0 - 4 = -6$
- $z_1 = x_0 - 0.15 \nabla_x g(x_0) = -0.1$

Projection step:

Since $z_1 = -0.1 < 0$, we project it to its closet point in feasible region $\mathcal{C}$

$$x_1 \leftarrow \Pi_{\mathcal{C}}(-0.1) = 0$$

Projected gradient descent: Example

Qn: Minimize $g(x) = (x - 2)^2$.

Subject to: $x \geq 0$ (feasible region $\mathcal{C} = [0, +\infty)$)

Initialize $x_0 = -1$, learning rate $\eta = 0.15$

1. Compute the gradient: $\nabla_x g(x) = 2x - 4$
2. At each iteration: $x_{k+1} \leftarrow x_k - \eta \nabla_x g(x_k)$ // gradient step (tentative)
 $\mathbf{x}_{k+1} \leftarrow \Pi_{\mathcal{C}}(\mathbf{z}_{k+1})$ // projection step

Iteration 2

Gradient step (tentative):

- $x_1 = 0$
- $\nabla_x g(x_1) = 2x_1 - 4 = -4$
- $z_2 = x_1 - 0.15 \nabla_x g(x_1) = 0.6$

Projection step:

Since $z_2 = 0.6 \geq 0$, its projection is itself

$$x_2 \leftarrow \Pi_{\mathcal{C}}(0.6) = 0.6$$

Projected gradient descent: Example

Qn: Minimize $g(x) = (x - 2)^2$.

Subject to: $x \geq 0$ (feasible region $\mathcal{C} = [0, +\infty)$)

Initialize $x_0 = -1$, learning rate $\eta = 0.15$

1. Compute the gradient: $\nabla_x g(x) = 2x - 4$
2. At each iteration: $x_{k+1} \leftarrow x_k - \eta \nabla_x g(x_k)$ // gradient step (tentative)
 $\mathbf{x}_{k+1} \leftarrow \Pi_{\mathcal{C}}(\mathbf{z}_{k+1})$ // projection step

Iteration 3

Gradient step (tentative):

- $x_2 = 0.6$
- $\nabla_x g(x_2) = 2x_2 - 4 = -2.8$
- $z_3 = x_2 - 0.15 \nabla_x g(x_2) = 1.02$

Projection step:

Since $z_3 = 1.02 \geq 0$, its projection is itself
 $x_3 \leftarrow \Pi_{\mathcal{C}}(1.02) = 1.02$

After 3 iterations, we are at 1.02, continuously approaching the optimal solution at $x=2$.

Summary

